



**ADDIS ABABA SCIENCE AND TECHNOLOGY
UNIVERSITY**

**COLLEGE OF SOCIAL AND NATURAL
SCIENCE**

**SOFTWARE ENGINEERING
DEPARTMENT**

**SWEG4103 Mobile computing and programming
1 st Semester 2025/2026**

Enbla Mobile App

Software Requirements Specification (SRS)

Name: Dawit Tadele

ID: ETS 0375/15

Section: B

Submitted to: **Inst. GetenetAssefa**

Submission Date: **Jun 16, 2026**

Table of Contents

1. Introduction	1
1.1 Project Overview	1
1.2 Purpose	1
1.3 Scope	1
1.4 Document Conventions	1
2. Overall Description	2
2.1 Product Perspective	2
2.2 User Classes and Characteristics	2
2.2.1 End Customer	2
2.2.2 Restaurant Manager	2
2.3 Operating Environment	3
2.4 Design and Implementation Constraints	3
3. System Features	4
3.1 Authentication System	4
3.1.1 Sign Up	4
3.1.2 Login	5
3.1.3 Logout	6
3.2 Customer Features	6
3.2.1 Customer Home Screen	6
3.2.2 Restaurant Details Screen	7
3.2.3 Reservation Form Screen	7
3.2.4 My Reservations Screen	8
3.3 Manager Features	9
3.3.1 Manager Home Screen	9
3.3.2 Add Restaurant Screen	10
3.3.3 Edit Restaurant Screen	10
3.3.4 Manager Reservations Screen	11
3.4 Profile Management	12
3.4.1 Customer Profile Screen	12
3.4.2 Manager Profile Screen	12
4. External Interface Requirements	13
4.1 User Interfaces	13
4.1.1 Screen Flow	13
4.1.2 Color Scheme and Design	13
4.2 Hardware Interfaces	14
4.3 Software Interfaces	14
4.3.1 Firebase Authentication API	14
4.3.2 Firestore Database API	14

4.3.3 Supabase Storage API	15
4.4 Communication Interfaces	15
5. System Features and Requirements	16
5.1 Data Requirements	16
5.1.1 User Data	16
5.1.2 Restaurant Data	16
5.1.3 Reservation Data	17
5.2 Performance Requirements	17
5.3 Security Requirements	17
5.4 Reliability and Availability	18
5.5 Usability Requirements	18
6. Non-Functional Requirements	19
6.1 Scalability	19
6.2 Maintainability	19
6.3 Portability	19
6.4 Compliance	19
7. Acceptance Criteria	20
8. Glossary	21
9. Appendices	22
9.1 Technology Stack Summary	22
9.2 Development Timeline	22
9.3 Future Enhancements	22
10. Sign-Off	23

Enbla Restaurant Reservation Mobile Application

Using Flutter, Dart, Firebase and Supabase

Software Requirements Specification (SRS)

1. Introduction

1.1 Project Overview

Enbla is a mobile application designed to streamline restaurant reservations and management. It connects customers with restaurants and provides restaurant managers with tools to manage their establishments, handle reservations, and upload restaurant details including photos.

1.2 Purpose

This document specifies the functional and non-functional requirements for the Enbla application development using Flutter, Firebase Authentication, Firestore database, and Supabase Storage.

1.3 Scope

The application includes two main user roles:

- **Customers:** Browse restaurants, make reservations, manage bookings
- **Managers:** Add/edit restaurants, manage reservations, upload restaurant images

1.4 Document Conventions

- **MUST:** Mandatory requirement
- **SHOULD:** Recommended requirement
- **MAY:** Optional requirement
- **API:** Application Programming Interface
- **UI:** User Interface
- **UID:** User Identifier

2. Overall Description

2.1 Product Perspective

Enbla operates as a standalone mobile application with the following architecture:

- **Frontend:** Flutter (Dart) for iOS and Android
- **Authentication:** Firebase Authentication (Email/Password + Google OAuth)
- **Database:** Firestore for structured data
- **Storage:** Supabase Storage for restaurant images
- **Real-time Updates:** Firestore listeners for live data synchronization

2.2 User Classes and Characteristics

2.2.1 End Customer

- **Profile:** Individual looking to make restaurant reservations
- **Technical Skill:** Low to medium
- **Frequency:** Occasional to regular usage
- **Key Activities:** Browse restaurants, make reservations, view reservation status

2.2.2 Restaurant Manager

- **Profile:** Restaurant owner or staff member managing bookings
- **Technical Skill:** Low to medium
- **Frequency:** Daily usage
- **Key Activities:** Add restaurants, manage reservations, upload photos, edit restaurant info

2.3 Operating Environment

- **Mobile Platforms:** Android 8.0+, iOS 12.0+
- **Network:** Internet connectivity required (WiFi or mobile data)
- **Firebase Project:** enbla-app (or custom)
- **Supabase Project:** For image storage (public bucket)

2.4 Design and Implementation Constraints

- Single-file Flutter project structure for initial development
- HTTP/REST APIs through Firebase and Supabase

- No local database (all data cloud-based)
 - Free tier services: Firebase free tier, Supabase free tier
-

3. System Features

3.1 Authentication System

3.1.1 Sign Up Feature

Description: Allow new users to create an account as either Customer or Manager.

Functional Requirements:

- **FR-AUTH-1:** User **MUST** enter Name, Email, Password, and Confirm Password
- **FR-AUTH-2:** Email validation **MUST** check for valid email format
- **FR-AUTH-3:** Password **MUST** be minimum 6 characters
- **FR-AUTH-4:** Confirm Password **MUST** match Password field
- **FR-AUTH-5:** User **MUST** select registration type: Customer or Manager
- **FR-AUTH-6:** Duplicate email prevention: **MUST** reject if email already exists
- **FR-AUTH-7:** On successful signup, user document **MUST** be created in Firestore with fields: uid, email, name, role (customer/manager), createdAt, updatedAt
- **FR-AUTH-8:** System **MUST** display error messages for invalid inputs (weak password, email in use, network error)
- **FR-AUTH-9:** After signup, user **MUST** be automatically logged in
- **FR-AUTH-10:** User **SHOULD** be redirected to appropriate home screen based on role (Customer → Customer Home, Manager → Manager Home)

UI Components:

- Text input fields for Name, Email, Password, Confirm Password
- Role selection buttons (Customer / Manager)
- Sign Up button
- Link to Login screen
- Error message display area

Data Validation:

- Name: Non-empty, max 50 characters
- Email: Valid email format (RFC 5322)
- Password: Min 6 chars, alphanumeric
- Confirm Password: Exact match with Password

3.1.2 Login Feature

Description: Allow existing users to authenticate with email/password or Google.

Functional Requirements:

- **FR-AUTH-11:** User **MUST** enter Email and Password
- **FR-AUTH-12:** System **MUST** verify credentials against Firebase Authentication
- **FR-AUTH-13:** On successful login, user **MUST** be redirected based on role stored in Firestore
- **FR-AUTH-14:** Failed login **MUST** display error: "Invalid email or password"
- **FR-AUTH-15:** Google Sign-In button **MUST** authenticate via Firebase Google OAuth
- **FR-AUTH-16:** Google authentication **MUST** create/update user document in Firestore if new user
- **FR-AUTH-17:** User session **MUST** persist across app restarts (Firebase Auth handles)
- **FR-AUTH-18:** User **MUST** see option to navigate to Sign Up if not registered

UI Components:

- Email input field
- Password input field (masked)
- Login button
- Google Sign-In button with Google branding
- "Forgot Password" link (optional for future)
- Link to Sign Up screen

Authentication Flow:

1. User enters email + password → Firebase verifies
2. If valid → Retrieve role from Firestore → Redirect accordingly
3. If invalid → Display error message
4. Google OAuth: User taps "Sign with Google" → Google popup → Firebase handles token → Create/fetch user doc

3.1.3 Logout Feature

Description: Allow logged-in users to end their session.

Functional Requirements:

- **FR-AUTH-19:** Logout button **MUST** be visible on Customer Home and Manager Home screens

- **FR-AUTH-20:** Logout MUST clear Firebase session
 - **FR-AUTH-21:** After logout, user MUST be redirected to Login screen
 - **FR-AUTH-22:** All user data in memory MUST be cleared
-

3.2 Customer Features

3.2.1 Customer Home Screen

Description: Main screen displaying restaurants and navigation for customers.

Functional Requirements:

- **FR-CUST-1:** Screen MUST display greeting: "Hello, [Customer Name]"
- **FR-CUST-2:** Screen MUST show list of all restaurants with: Image, Name, Description
- **FR-CUST-3:** Each restaurant MUST be clickable to view details
- **FR-CUST-4:** Bottom navigation MUST have two tabs: Home (restaurant list) and Profile
- **FR-CUST-5:** Data MUST be loaded from Firestore collection `restaurants`
- **FR-CUST-6:** Images MUST be loaded from Supabase Storage (restaurant-images bucket)
- **FR-CUST-7:** System MUST handle missing/broken images gracefully (placeholder)
- **FR-CUST-8:** App MUST refresh restaurant list periodically or on user request

Data Source: Firestore `restaurants` collection

UI Components:

- AppBar with greeting and profile avatar
 - Scrollable list of restaurant cards (image + name + description + arrow)
 - Bottom navigation bar
-

3.2.2 Restaurant Details Screen

Description: Display full details of a selected restaurant and allow reservation.

Functional Requirements:

- **FR-CUST-9:** Screen MUST display restaurant: Photo (full width), Name, Location, Description, Manager Name
- **FR-CUST-10:** "Reserve Table" button MUST navigate to Reservation Form screen
- **FR-CUST-11:** Screen MUST show back button to return to home
- **FR-CUST-12:** All data MUST be fetched from Firestore `restaurants` document

UI Components:

- Back button (top left)
- Restaurant image
- Text fields for Name, Location, Description, Manager
- "Reserve Table" button

3.2.3 Reservation Form Screen

Description: Allow customer to make a reservation at a restaurant.

Functional Requirements:

- **FR-CUST-13:** Screen MUST display pre-filled restaurant name
- **FR-CUST-14:** Form MUST have fields: Name (pre-filled), Date, Time, Number of Guests
- **FR-CUST-15:** Date picker MUST allow selection of today or future dates only
- **FR-CUST-16:** Time picker MUST display 24-hour format
- **FR-CUST-17:** Number of Guests MUST be numeric input (min 1, max 50)
- **FR-CUST-18:** "Reserve" button MUST save reservation to Firestore `reservations` collection
- **FR-CUST-19:** Reservation document MUST contain: `restaurantId`, `customerId`, `customerName`, `location`, `date`, `time`, `noOfGuests`, `status` (default: "Requested"), `createdAt`
- **FR-CUST-20:** System MUST display success message after reservation
- **FR-CUST-21:** After success, user MUST be redirected to "My Reservations" screen

Data Structure (Firestore):

```
reservations/  
├── {reservationId}  
│   ├── restaurantId: string  
│   ├── customerId: string (uid)  
│   ├── customerName: string  
│   ├── location: string  
│   ├── date: timestamp  
│   ├── time: string (HH:MM format)  
│   ├── noOfGuests: number  
│   ├── status: string (Requested/Confirmed/Rejected)  
└── createdAt: timestamp
```

3.2.4 My Reservations Screen

Description: Display all reservations made by the logged-in customer.

Functional Requirements:

- **FR-CUST-22:** Screen MUST display greeting: "Hello, [Customer Name]"
- **FR-CUST-23:** Screen MUST list all reservations for current user with: Restaurant Name, Location, Date, Time, Status (Confirmed/Requested), Cancel button
- **FR-CUST-24:** Reservations MUST be fetched from Firestore where customerId = currentUser.uid
- **FR-CUST-25:** Status badges MUST show different colors: Confirmed (green), Requested (yellow), Rejected (red)
- **FR-CUST-26:** "Cancel" button MUST delete reservation with confirmation
- **FR-CUST-27:** Bottom navigation MUST show Home and Profile tabs
- **FR-CUST-28:** Profile section MUST display: Name, Email, Logout button

UI Components:

- AppBar with greeting
 - List of reservation cards (restaurant, location, date, time, status, cancel button)
 - Bottom navigation
 - Profile section
-

3.3 Manager Features

3.3.1 Manager Home Screen

Description: Main dashboard for restaurant managers.

Functional Requirements:

- **FR-MGR-1:** Screen MUST display greeting: "Hello, [Manager Name]"
- **FR-MGR-2:** Screen MUST show "My Restaurants" section with list of manager's restaurants
- **FR-MGR-3:** Each restaurant MUST show: Photo, Name, and navigation arrow
- **FR-MGR-4:** Screen MUST have "Add Restaurants" button
- **FR-MGR-5:** Data MUST be fetched from Firestore where manager.uid = currentUser.uid
- **FR-MGR-6:** Bottom navigation MUST have Home and Profile tabs
- **FR-MGR-7:** Clicking a restaurant MUST navigate to Restaurant Details (Manager view) or Edit screen

Data Source: Firestore restaurants collection (filtered by managerId)

UI Components:

- AppBar with greeting and profile avatar

- "My Restaurants" list with cards (image + name + arrow)
- "Add Restaurants" button
- Bottom navigation bar

3.3.2 Add Restaurant Screen

Description: Allow manager to add a new restaurant with details and photo.

Functional Requirements:

- **FR-MGR-8:** Form MUST have fields: Upload Photo, Name, Location, Description, Manager Name (pre-filled)
- **FR-MGR-9:** "Upload Photo" button MUST open image picker (gallery)
- **FR-MGR-10:** Selected image MUST be uploaded to Supabase Storage (restaurant-images bucket)
- **FR-MGR-11:** Image URL from Supabase MUST be stored in Firestore
- **FR-MGR-12:** Image preview MUST be shown after selection
- **FR-MGR-13:** "Add" button MUST save restaurant document to Firestore restaurants collection
- **FR-MGR-14:** Restaurant document MUST contain: name, location, description, managerId (uid), managerName, photoUrl, createdAt, updatedAt
- **FR-MGR-15:** All fields MUST be validated before saving (non-empty, etc.)
- **FR-MGR-16:** System MUST display success message and redirect to Manager Home

Data Structure (Firestore):

```
restaurants/
├── {restaurantId}
│   ├── name: string
│   ├── location: string
│   ├── description: string
│   ├── managerId: string (uid)
│   ├── managerName: string
│   ├── photoUrl: string (Supabase URL)
│   ├── createdAt: timestamp
│   └── updatedAt: timestamp
```

Image Upload Flow:

1. User taps "Upload Photo" → ImagePicker opens gallery
2. User selects image → Preview shown
3. On "Add" click → Image uploaded to Supabase Storage:
restaurants/{timestamp}_{filename}
4. Supabase returns public URL → Stored in Firestore document

3.3.3 Edit Restaurant Screen

Description: Allow manager to edit existing restaurant details and photo.

Functional Requirements:

- **FR-MGR-17:** Screen MUST display current restaurant data in form fields (pre-filled)
 - **FR-MGR-18:** Manager MUST be able to change Name, Location, Description, Photo
 - **FR-MGR-19:** "Upload Photo" button MUST open image picker to replace current photo
 - **FR-MGR-20:** Current photo MUST be displayed in preview
 - **FR-MGR-21:** "Update" button MUST save changes to Firestore
 - **FR-MGR-22:** Old photo in Supabase MAY be deleted (optional cleanup)
 - **FR-MGR-23:** System MUST update updatedAt timestamp
 - **FR-MGR-24:** Success message MUST be displayed, redirect to Manager Home
-

3.3.4 Manager Reservations Screen

Description: Display all reservations for manager's restaurants and allow acceptance/rejection.

Functional Requirements:

- **FR-MGR-25:** Screen MUST list all reservations for all restaurants owned by manager
- **FR-MGR-26:** Each reservation MUST show: Restaurant Name, Location, Date, Time, Customer Name, No. of Guests, Status
- **FR-MGR-27:** Reservations MUST be fetched from Firestore where restaurantId IN (manager's restaurant IDs)
- **FR-MGR-28:** For "Requested" status reservations, MUST show "Accept" and "Reject" buttons
- **FR-MGR-29:** "Accept" button MUST update reservation status to "Confirmed"
- **FR-MGR-30:** "Reject" button MUST update reservation status to "Rejected"
- **FR-MGR-31:** Status badges MUST show different colors: Confirmed (green), Requested (yellow), Rejected (red)
- **FR-MGR-32:** After action, status MUST update immediately in UI

Data Updates:

- Status field in Firestore reservations document MUST be updated
 - Updated reservation SHOULD reflect in Customer's "My Reservations" in real-time
-

3.4 Profile Management

3.4.1 Customer Profile Screen

Description: Display and manage customer profile information.

Functional Requirements:

- **FR-PROF-1:** Screen MUST display: Name (from Firestore), Email (from Firebase Auth), Role badge
- **FR-PROF-2:** "My Reservation" navigation link MUST be clickable
- **FR-PROF-3:** "Log out" button MUST end session and return to Login screen
- **FR-PROF-4:** Profile data MUST be fetched from Firestore `users/{uid}` or Firebase Auth

UI Components:

- Avatar with user initial
 - Name, Email, Role
 - "My Reservation" button
 - "Log out" button
-

3.4.2 Manager Profile Screen

Description: Display and manage manager profile information.

Functional Requirements:

- **FR-PROF-5:** Screen MUST display: Name (from Firestore), Email (from Firebase Auth), Role badge (Manager)
 - **FR-PROF-6:** "Reservations" navigation link MUST lead to Manager Reservations screen
 - **FR-PROF-7:** "Log out" button MUST end session and return to Login screen
-
-

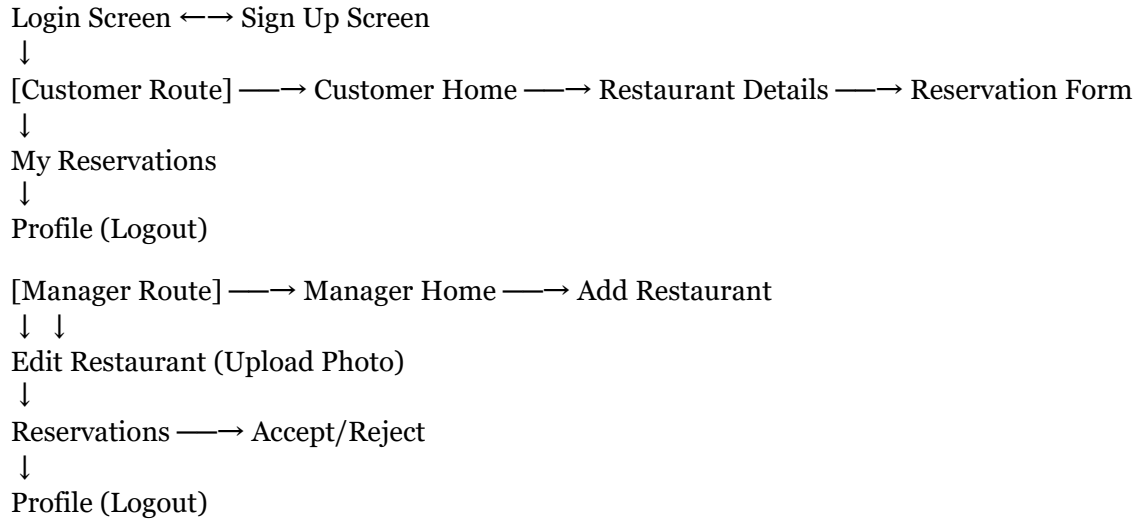
4. External Interface Requirements

4.1 User Interfaces

4.1.1 Screen Flow

Splash Screen





4.1.2 Color Scheme and Design

- **Primary Color:** Maroon/Brown (#5E5240, #7F6B55)
 - **Secondary Color:** Teal/Cyan (#32B8C6)
 - **Accent Color:** Light Cream (#FAFAF9)
 - **Text Color:** Dark Charcoal (#1F2121)
 - **Success Status:** Green
 - **Pending Status:** Yellow
 - **Rejected Status:** Red
 - **Font:** Material Design system fonts
 - **Layout:** Responsive for mobile screens
-

4.2 Hardware Interfaces

- **Device:** Android 8.0+, iOS 12.0+
 - **Display:** Minimum 4.5-inch screen
 - **Camera:** Accessible for image upload (optional, uses gallery picker)
-

4.3 Software Interfaces

4.3.1 Firebase Authentication API

Endpoint: Firebase SDK (firebase_auth package)

Methods:

- `createUserWithEmailAndPassword(email, password)` - Sign up
 - `signInWithEmailAndPassword(email, password)` - Login
 - `signInWithGoogle()` - Google OAuth
 - `signOut()` - Logout
 - `currentUser` - Get logged-in user
 - `onAuthStateChanged()` - Listen to auth state
-

4.3.2 Firestore Database API

Collections:

- `users/{uid}` - User profiles
- `restaurants/{restaurantId}` - Restaurant data
- `reservations/{reservationId}` - Reservation records

Operations:

- `create()` - Add new document
 - `read()` - Fetch document
 - `update()` - Modify document
 - `delete()` - Remove document
 - `query()` - Filter/search
-

4.3.3 Supabase Storage API

Bucket: `restaurant-images` (public)

Methods:

- `upload(path, file)` - Upload image
 - `getPublicUrl(path)` - Get public URL
 - `remove(path)` - Delete image (optional)
-

4.4 Communication Interfaces

- **Network:** HTTPS/REST APIs
- **Authentication:** Firebase tokens, API keys
- **Data Format:** JSON
- **Real-time Updates:** Firestore listeners (Web Socket)

5. System Features and Requirements

5.1 Data Requirements

5.1.1 User Data

```
users/  
├── {uid}  
│   ├── uid: string (Firebase UID)  
│   ├── email: string (unique)  
│   ├── name: string  
│   ├── role: string (customer | manager)  
│   ├── createdAt: timestamp  
│   └── updatedAt: timestamp
```

5.1.2 Restaurant Data

```
restaurants/  
├── {restaurantId}  
│   ├── restaurantId: string (auto-generated)  
│   ├── name: string  
│   ├── location: string  
│   ├── description: string  
│   ├── managerId: string (reference to user)  
│   ├── managerName: string  
│   ├── photoUrl: string (Supabase URL)  
│   ├── createdAt: timestamp  
│   └── updatedAt: timestamp
```

5.1.3 Reservation Data

```
reservations/  
├── {reservationId}  
│   ├── reservationId: string (auto-generated)  
│   ├── restaurantId: string (reference)  
│   ├── customerId: string (reference)  
│   ├── customerName: string  
│   ├── location: string  
│   ├── date: timestamp  
│   ├── time: string (HH:MM)  
│   ├── noOfGuests: number (1-50)  
│   ├── status: string (Requested | Confirmed | Rejected)  
│   └── createdAt: timestamp
```

5.2 Performance Requirements

- **FR-PERF-1:** App startup time MUST be under 3 seconds
 - **FR-PERF-2:** Restaurant list load time MUST be under 2 seconds (with caching)
 - **FR-PERF-3:** Image load time MUST not block UI (lazy loading)
 - **FR-PERF-4:** Search/filter operations MUST respond within 1 second
 - **FR-PERF-5:** Real-time updates MUST reflect within 2 seconds
-

5.3 Security Requirements

- **FR-SEC-1:** Passwords MUST be minimum 6 characters (Firebase handles hashing)
- **FR-SEC-2:** Sensitive data (passwords, tokens) MUST NOT be stored in plaintext
- **FR-SEC-3:** HTTPS MUST be used for all network communications
- **FR-SEC-4:** Firestore security rules MUST enforce:
 - Users can only read their own documents
 - Managers can only update their own restaurants
 - Customers can only view public restaurant data
- **FR-SEC-5:** API keys (Supabase anon key) MUST be kept in environment files
- **FR-SEC-6:** Firebase tokens MUST be automatically refreshed
- **FR-SEC-7:** Logout MUST clear all cached user data

Firestore Security Rules (Example):

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Users collection
    match /users/{uid} {
      allow read, write: if request.auth.uid == uid;
    }

    // Restaurants collection
    match /restaurants/{restaurantId} {
      allow read: if true; // Public read
      allow create, update, delete: if request.auth.uid ==
resource.data.managerId;
    }

    // Reservations collection
    match /reservations/{reservationId} {
      allow read: if request.auth.uid == resource.data.customerId
        || request.auth.uid == resource.data.managerId;
      allow create: if request.auth.uid == request.resource.data.customerId;
      allow update: if request.auth.uid == resource.data.managerId;
```

```
}  
  
}  
}
```

5.4 Reliability and Availability

- **FR-REL-1:** App MUST handle network disconnection gracefully
 - **FR-REL-2:** Failed requests MUST be retried automatically (max 3 attempts)
 - **FR-REL-3:** Error messages MUST be user-friendly and actionable
 - **FR-REL-4:** App MUST not crash on invalid data
 - **FR-REL-5:** Firestore operations MUST have timeout handling (30 seconds)
-

5.5 Usability Requirements

- **FR-USE-1:** UI MUST follow Material Design guidelines
 - **FR-USE-2:** All interactive elements MUST be at least 48×48 dp in size
 - **FR-USE-3:** Color contrast MUST meet WCAG AA standards (4.5:1 for text)
 - **FR-USE-4:** App MUST support both portrait and landscape orientations
 - **FR-USE-5:** Navigation MUST be intuitive with clear back buttons
 - **FR-USE-6:** Form validation errors MUST be clear and specific
-

6. Non-Functional Requirements

6.1 Scalability

- App MUST support 1000+ concurrent users
 - Database queries MUST use indexes for performance
 - Images MUST be optimized (compressed) before upload
-

6.2 Maintainability

- Code MUST follow Dart/Flutter best practices
- Comments MUST document complex logic
- Modular architecture MUST separate UI, logic, and data layers

- Version control (Git) MUST be used for all changes
-

6.3 Portability

- App MUST compile and run on Android and iOS
 - Codebase MUST not contain platform-specific code (use Flutter plugins)
-

6.4 Compliance

- App MUST comply with app store policies (Google Play, Apple App Store)
 - User data MUST be handled according to privacy regulations (GDPR, etc.)
 - Terms of Service and Privacy Policy MUST be displayed to users
-
-

7. Acceptance Criteria

7.1 Authentication Module Acceptance

- User can successfully sign up with email/password
 - User can successfully sign up with Google
 - User can login with existing credentials
 - User can logout and session is cleared
 - Duplicate emails are rejected
 - Invalid password formats are rejected
 - Appropriate error messages are displayed
-

7.2 Customer Features Acceptance

- Customer can view restaurant list with images
 - Customer can view restaurant details
 - Customer can make a reservation
 - Customer can view and cancel reservations
 - Reservation status updates reflect in real-time
 - Customer profile displays correct information
-

7.3 Manager Features Acceptance

- Manager can add new restaurant with photo
 - Manager can edit restaurant details
 - Manager can upload and replace restaurant images
 - Manager can view all reservations for their restaurants
 - Manager can accept/reject reservations
 - Reservation status changes are visible to customers in real-time
-

7.4 Data Persistence Acceptance

- All user data persists in Firestore
 - Restaurant images are accessible from Supabase
 - Reservations are stored and retrievable
 - Updates are reflected across all instances of the app
-

7.5 Performance Acceptance

- App launches in under 3 seconds
 - Restaurant list loads in under 2 seconds
 - Navigation between screens is smooth (no lag)
 - Images load without blocking UI
-
-

8. Glossary

Term	Definition
Customer	End-user who makes restaurant reservations
Manager	Restaurant owner/staff who manages reservations and restaurant info
Firebase Auth	Google's authentication service (email/password + OAuth)
Firestore	NoSQL database by Google for structured data
Supabase Storage	File storage service for images and documents
UID	Unique identifier assigned by Firebase Auth

OAuth	Open authentication protocol for secure login
Reservation	Booking request for a table at a restaurant
Status	State of a reservation (Requested, Confirmed, Rejected)
API	Application Programming Interface for system communication
Real-time	Instant data synchronization across app instances

9. Appendices

9.1 Technology Stack Summary

Component	Technology
Frontend	Flutter (Dart)
Authentication	Firebase Authentication
Database	Firestore (NoSQL)
Image Storage	Supabase Storage
IDE	Visual Studio Code
Version Control	Git / GitHub
Design Tool	Figma
Target Platforms	Android 8.0+, iOS 12.0+

9.2 Development Timeline (Suggested)

Phase	Duration	Tasks
Setup & Auth	2-3 days	Firebase setup, Authentication UI, Login/Signup
Customer Features	3-4 days	Home screen, Restaurant details, Reservations
Manager Features	3-4 days	Restaurant management, Image upload, Accept/Reject
Testing & Refinement	2-3 days	Bug fixes, Performance optimization, UI polish
Deployment	1-2 days	App store submission, Beta testing

9.3 Future Enhancements (Out of Scope)

- Payment integration (Stripe, PayPal)
- Email/SMS notifications
- Ratings and reviews
- Favorites/Wishlist
- Restaurant search and filters
- Advanced analytics
- Multi-language support
- Dark mode

10. Sign-Off

This SRS document defines the complete requirements for the Enbla Restaurant Reservation Application. All features, functional and non-functional requirements, and acceptance criteria are documented herein.

Document Version: 1.0

Last Updated: January 16, 2026

Status: Complete and Ready for Development

Prepared By: Dawit Tadele

